

Билет 10. Неблокирующие `read` и `write`. Busy loop polling. Проблемы такой реализации

0. Формулировка билета

Неблокирующие `read` и `write`. Развитие сервера с тред пулом: busy loop polling. Проблемы такой реализации.

Главная идея билета:

Блокирующий `read` усыпляет поток, пока данных нет. Поэтому в thread pool один медленный клиент может занять целый worker.

Неблокирующий `read` не усыпляет поток: если данных нет, он сразу возвращает ошибку `EAGAIN` / `EWOULDBLOCK`.

Тогда один поток может по очереди опрашивать много соединений. Это называется busy loop polling. Но такой подход жжёт CPU и плохо масштабируется, потому что приходится постоянно просматривать все fd.

Ключевая цепочка:

```
thread per client
  ↓
thread pool
  ↓
workers всё равно блокируются на read/write
  ↓
non-blocking sockets
  ↓
busy loop polling по всем client_fd
  ↓
проблемы: CPU burn, O(n), head-of-line blocking
  ↓
select/poll/epoll
```

1. Проблема блокирующего `read` в thread pool

В thread pool мы делали так:

```
pool.submit([client_fd] {  
    handle_client(client_fd);  
});
```

А внутри `handle_client` обычно было:

```
ssize_t n = read(client_fd, buffer, sizeof(buffer));
```

Если клиент подключился и ничего не отправляет, блокирующий `read` усыпляет worker-поток.

Пусть в пуле 4 worker-a:

```
worker #1 blocked on read(client #1)  
worker #2 blocked on read(client #2)  
worker #3 blocked on read(client #3)  
worker #4 blocked on read(client #4)
```

Если все 4 клиента молчат, пул занят полностью.

Новые клиенты могут быть приняты главным потоком, но их задачи будут лежать в очереди и не выполняться.

Проблема:

```
thread pool ограничил число потоков,  
но всё ещё не решил проблему большого числа медленных соединений.
```

Нужно сделать так, чтобы worker не блокировался на одном клиенте.

2. Blocking vs non-blocking I/O

Блокирующий `read`

Обычный сокет по умолчанию блокирующий.

```
ssize_t n = read(fd, buffer, sizeof(buffer));
```

Если данных нет:

```
поток засыпает в ядре  
поток не может обслуживать других клиентов
```

То есть поток ждёт именно этот fd.

Неблокирующий `read`

Если сокет переведён в неблокирующий режим, то:

```
ssize_t n = read(fd, buffer, sizeof(buffer));
```

если данных нет, не засыпает, а сразу возвращает:

```
n == -1  
errno == EAGAIN или EWOULDBLOCK
```

Смысл:

```
данных сейчас нет,  
но поток не блокируется,  
можешь заняться чем-то другим.
```

3. Как включить неблокирующий режим

Через `fcntl`:

```
#include <fcntl.h>  
#include <unistd.h>  
  
void set_nonblocking(int fd) {  
    int flags = fcntl(fd, F_GETFL, 0);  
  
    if (flags < 0) {  
        throw std::runtime_error("fcntl F_GETFL failed");  
    }  
  
    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0) {  
        throw std::runtime_error("fcntl F_SETFL failed");  
    }  
}
```

Использование:

```
int client_fd = accept(server_fd, nullptr, nullptr);
set_nonblocking(client_fd);
```

Можно сделать неблокирующим и `server_fd`, чтобы `accept` тоже не блокировался:

```
set_nonblocking(server_fd);
```

Тогда `accept` при отсутствии готовых соединений вернёт:

```
-1, errno == EAGAIN / EWOULDBLOCK
```

4. Неблокирующий `read`: варианты результата

```
ssize_t n = read(fd, buffer, sizeof(buffer));
```

Возможные случаи:

`n > 0`

Прочитали `n` байт.

```
if (n > 0) {
    // обработать buffer[0..n)
}
```

Важно: TCP — поток байтов, поэтому `read` может вернуть меньше, чем “полное сообщение”.

`n == 0`

EOF: другая сторона закрыла соединение на запись.

```
if (n == 0) {
    close(fd);
}
```

Для TCP это обычно значит, что клиент отключился.

```
n < 0 && errno == EAGAIN / EWOULDBLOCK
```

Данных сейчас нет.

```
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {  
    // не ошибка соединения  
    // просто сейчас нечего читать  
}
```

В неблокирующем режиме это нормальная ситуация.

```
n < 0 && errno == EINTR
```

Системный вызов прерван сигналом.

```
if (n < 0 && errno == EINTR) {  
    // можно повторить позже  
}
```

Другие ошибки

Например:

```
ECONNRESET  
EBADF  
EPIPE
```

Обычно соединение закрывают.

5. Неблокирующий `write`

`write` в неблокирующем режиме тоже не обязан записать всё.

```
ssize_t n = write(fd, data, size);
```

Возможные случаи:

```
n > 0
```

Записали `n` байт.

Но:

```
n может быть меньше size
```

Значит, остаток надо сохранить и дописать позже.

```
n < 0 && errno == EAGAIN / EWOULDBLOCK
```

Сейчас socket send buffer заполнен.

```
ядро не готово принять новые байты на отправку  
попробуй позже
```

Это не фатальная ошибка.

```
n < 0 && errno == EPIPE
```

Другая сторона закрыла соединение.

Также возможен `SIGPIPE`, если писать через `write` в закрытый сокет. В реальном коде используют `send(..., MSG_NOSIGNAL)` или игнорируют `SIGPIPE`.

6. Почему неблокирующий `write` сложнее, чем кажется

В blocking-сервере можно было написать:

```
write_all(fd, response.data(), response.size());
```

и поток будет ждать, пока всё отправится.

В non-blocking-сервере так нельзя.

Если `write` отправил только часть:

```
response = "hello world"  
write отправил только "hello"
```

то остаток:

```
" world"
```

надо сохранить в состоянии соединения.

Поэтому у каждого клиента появляется состояние:

```
struct Connection {  
    int fd;  
  
    std::string input_buffer;  
    std::string output_buffer;  
  
    bool closed = false;  
};
```

Если `output_buffer` не пустой, значит нам нужно продолжать писать в этот fd позже.

7. Неблокирующий сервер как автомат состояний

Блокирующий код выглядит линейно:

```
read request  
process request  
write response
```

Нeblikирующий код превращается в автомат:

```
есть данные для чтения?  
    read сколько получится  
    добавить в input_buffer  
    если собрали полный запрос:  
        обработать  
        положить ответ в output_buffer  
  
можно писать?  
    write сколько получится  
    удалить отправленную часть из output_buffer  
    если всё отправили:  
        ждать следующий запрос или закрыть
```

Состояние клиента хранится между итерациями цикла.

Пример структуры:

```
struct Connection {  
    int fd = -1;  
  
    std::string in;  
    std::string out;
```

```
bool want_close = false;
};
```

Это важный переход:

```
blocking I/O:
    состояние живёт на стеке потока

non-blocking I/O:
    состояние соединения надо хранить явно
```

8. Busy loop polling

Самая простая идея после non-blocking fd:

Держим список всех клиентов и в бесконечном цикле пробуем читать/писать из каждого.

Схема:

```
clients = [fd1, fd2, fd3, fd4, ...]

while true:
    асепт новых клиентов, если есть

    for each client in clients:
        try read(client)
        try write(client)
        close if disconnected
```

Это и есть busy loop polling.

Поток не блокируется на одном клиенте. Он постоянно обходит всех.

9. Простейший busy loop server

Упрощённый пример:

```
#include <arpa/inet.h>
#include <cerrno>
#include <cstring>
#include <fcntl.h>
#include <iostream>
```



```

#include <string>
#include <unistd.h>
#include <vector>
#include <sys/socket.h>

struct Connection {
    int fd;
    std::string out;
    bool closed = false;
};

void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);

    if (flags < 0) {
        throw std::runtime_error("fcntl F_GETFL failed");
    }

    if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0) {
        throw std::runtime_error("fcntl F_SETFL failed");
    }
}

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(11000);
    addr.sin_addr.s_addr = INADDR_ANY;

    bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
    listen(server_fd, 128);

    set_nonblocking(server_fd);

    std::vector<Connection> clients;

    while (true) {
        // 1. Принимаем все готовые соединения
        while (true) {
            int client_fd = accept(server_fd, nullptr, nullptr);

            if (client_fd < 0) {
                if (errno == EAGAIN || errno == EWOULDBLOCK) {
                    break; // новых соединений сейчас нет
                }
            }
        }
    }
}

```

```

        if (errno == EINTR) {
            continue;
        }

        std::cerr << "accept failed: " << strerror(errno) << "\n";
        break;
    }

    set_nonblocking(client_fd);
    clients.push_back(Connection{client_fd});
}

// 2. Обходим всех клиентов
for (auto& client : clients) {
    char buffer[4096];

    while (true) {
        ssize_t n = read(client.fd, buffer, sizeof(buffer));

        if (n > 0) {
            // echo: прочитанное добавляем в очередь на отправку
            client.out.append(buffer, buffer + n);
            continue;
        }

        if (n == 0) {
            client.closed = true;
            break;
        }

        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            break; // сейчас данных больше нет
        }

        if (errno == EINTR) {
            continue;
        }

        client.closed = true;
        break;
    }

    // 3. Пишем, если есть что писать
    while (!client.out.empty()) {
        ssize_t n = write(
            client.fd,
            client.out.data(),
            client.out.size()
        );

        if (n > 0) {

```

```

        client.out.erase(0, static_cast<size_t>(n));
        continue;
    }

    if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
        break; // socket buffer заполнен, попробуем позже
    }

    if (n < 0 && errno == EINTR) {
        continue;
    }

    client.closed = true;
    break;
}

// 4. Удаляем закрытые соединения
clients.erase(
    std::remove_if(
        clients.begin(),
        clients.end(),
        [](const Connection& c) {
            if (c.closed) {
                close(c.fd);
                return true;
            }

            return false;
        }
    ),
    clients.end()
);
}
}

```

Это учебный код, но он показывает идею.

10. Что улучшилось по сравнению с thread pool

В thread pool:

один worker может заблокироваться на одном client_fd

В busy loop:

один поток не блокируется на read/write
он обходит много клиентов
если у клиента нет данных – идёт дальше

Плюс:

можно обслуживать много idle-соединений малым числом потоков
низкая задержка, если цикл крутится постоянно
нет context switch при ожидании каждого клиента

Именно поэтому busy polling иногда используют в low-latency системах.

11. Проблема 1: CPU burn

Busy loop постоянно крутится:

```
while (true) {  
    for (auto& client : clients) {  
        read(client.fd, ...);  
        write(client.fd, ...);  
    }  
}
```

Даже если никто ничего не отправляет, поток всё равно делает работу:

обходит clients
делает read
получает EAGAIN
идёт дальше
снова обходит clients
снова EAGAIN
...

Минус:

CPU загружен почти на 100%
большая часть работы холостая
энергопотребление растёт
другие потоки получают меньше CPU

Можно вставить:

```
std::this_thread::yield();
```

или:

```
std::this_thread::sleep_for(1ms);
```

но это ухудшает latency: можно проспать момент, когда данные появились.

12. Проблема 2: $O(n)$ polling

Пусть у нас 10 000 клиентов.

В каждой итерации мы проверяем всех:

```
fd1: EAGAIN  
fd2: EAGAIN  
fd3: EAGAIN  
...  
fd9999: EAGAIN  
fd10000: есть данные
```

Чтобы найти одного активного клиента, мы сделали 10 000 проверок.

Сложность:

```
 $O(\text{number\_of\_connections})$  на одну итерацию
```

Хотелось бы:

```
спросить ядро: дай мне только те fd, которые готовы
```

Это как раз делают:

```
select  
poll  
epoll
```

Следующий билет будет про это.

13. Проблема 3: много лишних системных вызовов

Каждый `read` по неготовому fd — это всё равно системный вызов.

Да, он быстро возвращает `EAGAIN`, но он не бесплатный.

Если клиентов много, а активных мало:

почти все `syscalls` возвращают `EAGAIN`

Это плохая модель.

В идеале поток должен спать, пока хотя бы один fd не станет готовым.

14. Проблема 4: head-of-line blocking внутри цикла

Представим:

client #1 прислал огромный запрос
client #2 прислал маленький запрос
client #3 ждёт быстрый ответ

Если обработчик `client #1` долго работает, остальные ждут, потому что всё в одном event loop.

Даже без blocking `read` можно заблокировать цикл долгой CPU-обработкой:

```
process_request(client); // может быть долгим
```

Проблема:

один долгий handler задерживает всех остальных

Это называется head-of-line blocking.

Решение в более зрелой архитектуре:

event loop занимается только I/O readiness
тяжёлую CPU-работу отдаёт в thread pool

15. Проблема 5: нужно хранить состояние соединения

В blocking-модели можно было писать код линейно:

```
read request
process
write response
```

В non-blocking-модели так нельзя.

Нужно хранить:

```
частично прочитанный запрос
частично отправленный ответ
состояние парсера протокола
хочет ли соединение закрыться
нужно ли сейчас писать
```

Например:

```
struct Connection {
    int fd;
    std::string input;
    std::string output;
    bool request_complete;
    bool close_after_write;
};
```

Это делает код сложнее.

16. Проблема 6: partial read/write и framing

TCP — поток байтов.

`read` может вернуть:

```
половину сообщения
несколько сообщений сразу
пустоту через EAGAIN
```

`write` может записать:

только часть ответа

Поэтому протокол должен иметь framing:

разделитель: `\n`
или `length-prefix`
или фиксированный размер

Пример `length-prefix`:

`[4 байта длины][payload]`

Сервер должен уметь хранить неполные данные в `input_buffer`.

17. Проблема 7: fairness

Если в цикле всегда идти от начала массива:

```
for (auto& client : clients) {  
    ...  
}
```

клиенты в начале могут получать немного более быстрый сервис.

Если один клиент постоянно активен, он может забирать много времени.

Нужно думать о fairness:

лимит чтения за одну итерацию
лимит обработки на клиента
`round-robin`
перенос активных клиентов в очередь

В настоящих event loop такие вещи учитываются.

18. Проблема 8: backpressure

Если клиент читает медленно, наш `output_buffer` может расти:

сервер генерирует ответы быстрее,
чем клиент их забирает

В blocking-модели `write` просто заблокировался бы.

В non-blocking-модели `write` возвращает `EAGAIN`, а данные остаются в `output_buffer`.

Если не ограничивать размер буфера:

память может расти бесконечно

Нужна политика:

лимит `output_buffer`
закрыть медленного клиента
приостановить чтение новых запросов от него
применить `backpressure` выше по протоколу

19. Busy loop polling vs `select/poll/epoll`

Busy loop polling:

программа сама проверяет каждый `fd`

`select/poll/epoll`:

ядро сообщает программе, какие `fd` готовы

Сравнение:

Подход	Что происходит
Busy loop	постоянно обходим все <code>fd</code>
<code>select</code>	передаём набор <code>fd</code> ядру, оно говорит готовые
<code>poll</code>	похож на <code>select</code> , но без <code>fd_set</code> -ограничения
<code>epoll</code>	регистраруем <code>fd</code> один раз, потом получаем готовые события

Главная проблема busy loop:

мы спрашиваем каждого клиента: "ты готов?"

Главная идея epoll:

ядро само скажет, кто готов

20. Неблокирующий `accept`

Если `server_fd` неблокирующий:

```
while (true) {
    int client_fd = accept(server_fd, nullptr, nullptr);

    if (client_fd < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            break;
        }

        if (errno == EINTR) {
            continue;
        }

        // настоящая ошибка
        break;
    }

    set_nonblocking(client_fd);
    clients.push_back(Connection{client_fd});
}
```

Почему `accept` в цикле?

Потому что в ассепт-очереди может лежать несколько готовых соединений.

Мы хотим принять все доступные прямо сейчас, пока `accept` не скажет:

EAGAIN: больше готовых соединений нет

21. Неполноценное чтение: читать до `EAGAIN`

Для одного fd обычно читают в цикле:

```

while (true) {
    ssize_t n = read(fd, buffer, sizeof(buffer));

    if (n > 0) {
        input.append(buffer, buffer + n);
        continue;
    }

    if (n == 0) {
        close_connection();
        break;
    }

    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        break;
    }

    if (errno == EINTR) {
        continue;
    }

    close_connection();
    break;
}

```

Почему до `EAGAIN`?

Потому что раз уж мы добрались до `fd`, можно вычитать всё, что уже доступно в kernel buffer.

Для `epoll` edge-triggered это будет особенно важно: если не вычитать до `EAGAIN`, нового события может не прийти.

22. Неблокирующая запись: писать до `EAGAIN`

Если есть `output_buffer`:

```

while (!output.empty()) {
    ssize_t n = write(fd, output.data(), output.size());

    if (n > 0) {
        output.erase(0, n);
        continue;
    }

    if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
        break;
    }
}

```

```
    if (n < 0 && errno == EINTR) {  
        continue;  
    }  
  
    close_connection();  
    break;  
}
```

Если `output` остался непустым:

дописать позже

В `epoll` для этого обычно подписываются на событие готовности к записи:

`EPOLLOUT`

Но в `busy loop` мы просто попробуем снова на следующем проходе.

23. Non-blocking не равно async

Важно различать:

Non-blocking

операция не ждёт
если не готово — сразу возвращает `EAGAIN`

Ты сам должен потом попробовать снова.

Async

ты запускаешь операцию
она завершается позже
результат приходит через `callback/future/completion event`

`epoll` — это не совсем `async I/O`, а `readiness notification`:

`fd` готов к чтению/записи
теперь сам вызови `read/write`

`io_uring` ближе к `completion`-модели:

операция была выполнена
в completion queue пришёл результат

24. Что сказать на экзамене: короткая версия

Можно отвечать так:

В блокирующем сервере `read` усыпляет поток, если данных нет. Поэтому в thread pool медленный клиент может занять целый worker. Если все workers ждут на `read`, новые клиенты не обслуживаются. Чтобы этого избежать, сокеты переводят в неблокирующий режим через `fcntl(fd, F_SETFL, flags | O_NONBLOCK)`.

В неблокирующем режиме `read` и `write` не блокируют поток. Если данных нет или socket buffer не готов, они возвращают `-1` с `errno == EAGAIN` или `EWOULDBLOCK`. Для `read`: `n > 0` — прочитали байты, `n == 0` — клиент закрыл соединение, `EAGAIN` — данных сейчас нет. Для `write`: можно записать только часть буфера, а при `EAGAIN` нужно сохранить остаток и дописать позже.

Самое простое развитие сервера — busy loop polling: храним список всех client fd и в бесконечном цикле пытаемся делать non-blocking `read/write` для каждого. Если данных нет, получаем `EAGAIN` и идём к следующему клиенту. Так один поток может обслуживать много соединений и не блокироваться на одном клиенте.

Проблемы busy loop: поток постоянно крутится и жжёт CPU; каждый проход стоит $O(n)$ по числу соединений; много лишних syscalls возвращают `EAGAIN`; один долгий handler задерживает остальных; нужно явно хранить состояние каждого соединения, input/output buffers и частичные read/write; возникает backpressure на медленных клиентах.

Поэтому busy loop polling — переходный учебный шаг. Настоящее решение — I/O multiplexing: `select`, `poll`, `epoll`, где ядро сообщает, какие fd готовы, и нам не нужно постоянно опрашивать все соединения вручную.

25. Мини-код для ответа на доске

```
void set_nonblocking(int fd) {
    int flags = fcntl(fd, F_GETFL, 0);
    fcntl(fd, F_SETFL, flags | O_NONBLOCK);
}
```

```
struct Connection {
    int fd;
    std::string in;
    std::string out;
```

```
    bool closed = false;
};
```

```
while (true) {
    // accept all ready clients
    while (true) {
        int client_fd = accept(server_fd, nullptr, nullptr);

        if (client_fd < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                break;
            }

            if (errno == EINTR) {
                continue;
            }

            break;
        }

        set_nonblocking(client_fd);
        clients.push_back(Connection{client_fd});
    }

    for (auto& c : clients) {
        // read until EAGAIN
        while (true) {
            char buf[4096];
            ssize_t n = read(c.fd, buf, sizeof(buf));

            if (n > 0) {
                c.in.append(buf, buf + n);
                c.out.append(buf, buf + n); // echo
                continue;
            }

            if (n == 0) {
                c.closed = true;
                break;
            }

            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                break;
            }

            if (errno == EINTR) {
                continue;
            }

            c.closed = true;
        }
    }
}
```

```

        break;
    }

    // write until EAGAIN
    while (!c.out.empty()) {
        ssize_t n = write(c.fd, c.out.data(), c.out.size());

        if (n > 0) {
            c.out.erase(0, n);
            continue;
        }

        if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
            break;
        }

        if (n < 0 && errno == EINTR) {
            continue;
        }

        c.closed = true;
        break;
    }
}
}

```

Что объяснить:

O_NONBLOCK – fd не блокирует поток
 read/write возвращают EAGAIN, если сейчас не готовы
 нужно хранить per-connection state
 busy loop постоянно опрашивает все fd
 это low-latency, но CPU-heavy и O(n)
 следующий шаг – select/poll/epoll

26. Мини-проверка

1. Чем блокирующий `read` отличается от неблокирующего?
2. Как перевести fd в неблокирующий режим?
3. Что означает `read == -1` и `errno == EAGAIN`?
4. Что означает `read == 0`?
5. Почему неблокирующий `write` может записать только часть буфера?
6. Почему в non-blocking сервере нужен `output_buffer` на клиента?
7. Что такое busy loop polling?
8. Чем busy loop лучше thread pool с blocking `read`?
9. Почему busy loop жжёт CPU?
10. Почему busy loop имеет проблему O(n)?

11. Почему один долгий обработчик может задержать всех клиентов?
12. Почему следующий шаг после busy loop — `select/poll/epoll` ?